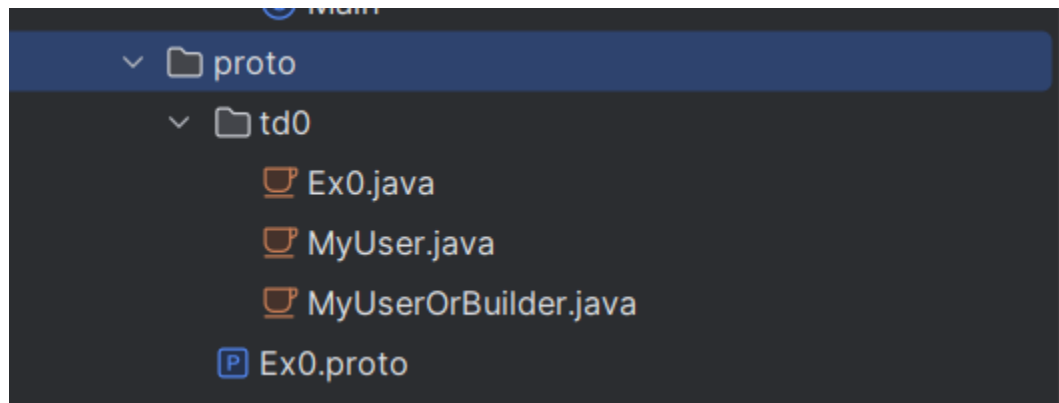


TD gRPC

```
could not make proto path relative: Ex0.proto: no such file or directory
PS C:\Users\User\Desktop\grpc\grpcLab> cd C:\Users\User\Desktop\grpc\grpcLab\src\main\proto
PS C:\Users\User\Desktop\grpc\grpcLab\src\main\proto> protoc -I . --java_out . Ex0.proto
PS C:\Users\User\Desktop\grpc\grpcLab\src\main\proto> karen khoury
```

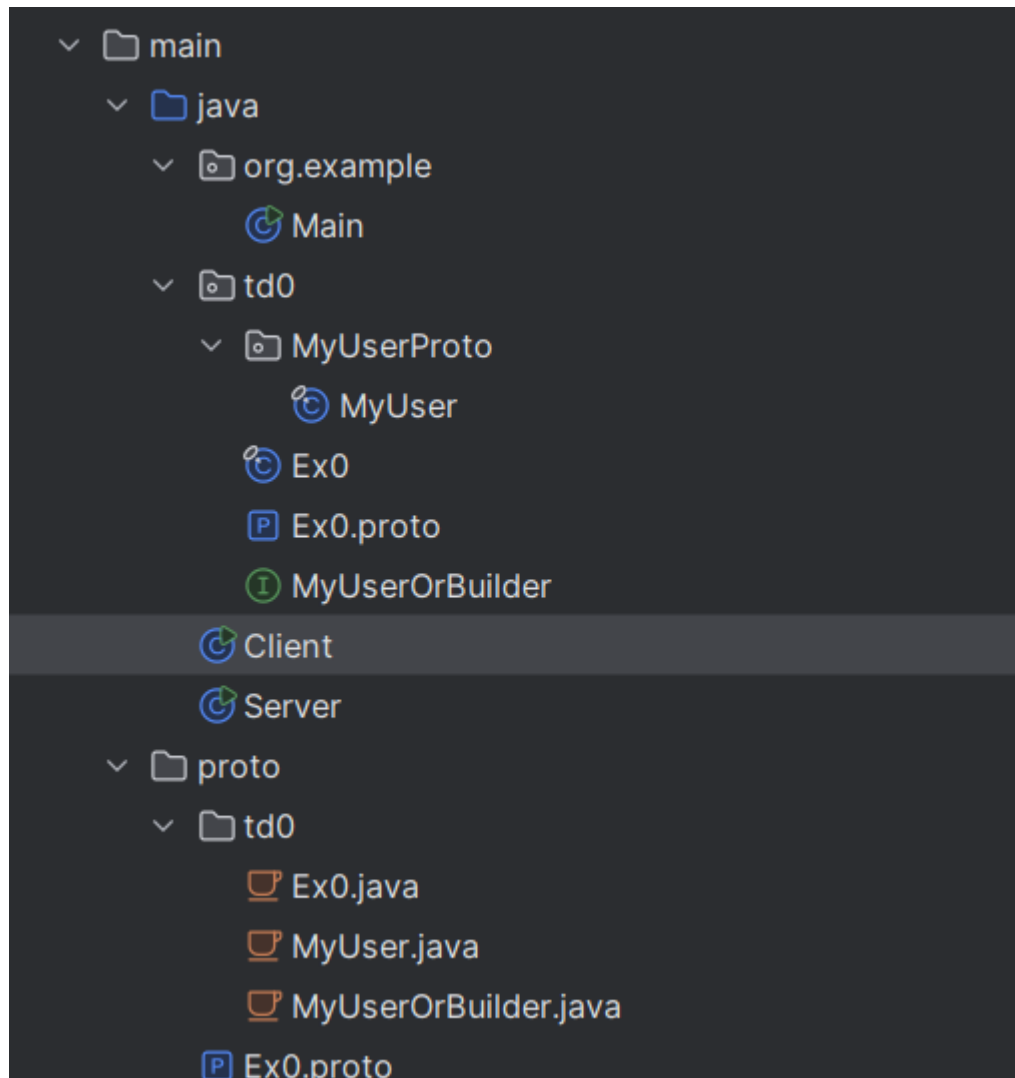


MyUser.newBuilder() pour créer un objet MyUser

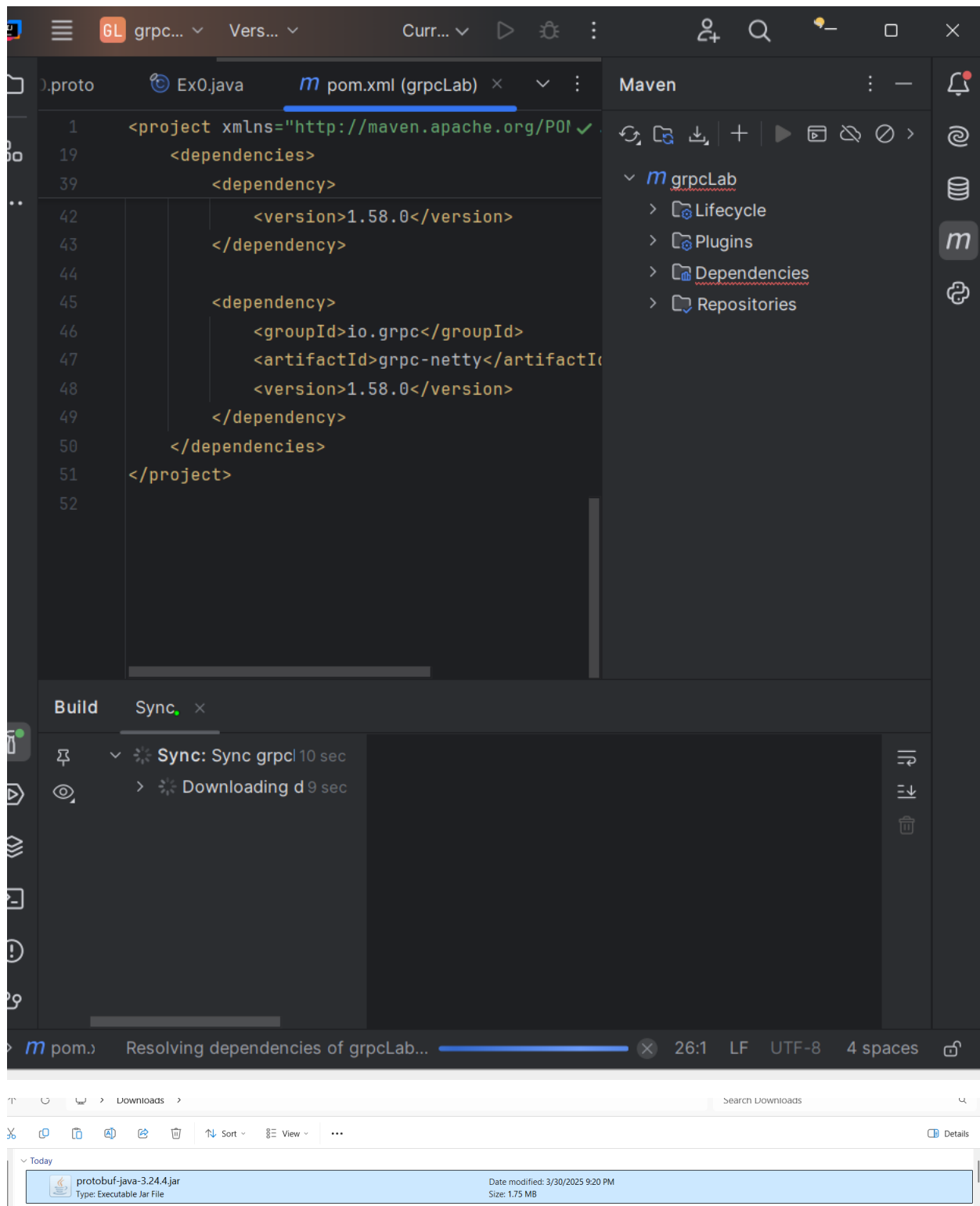
La classe principale : my user

et le fichier ex0.java : - La description du fichier proto

et les informations de réflexion utiles à gRPC

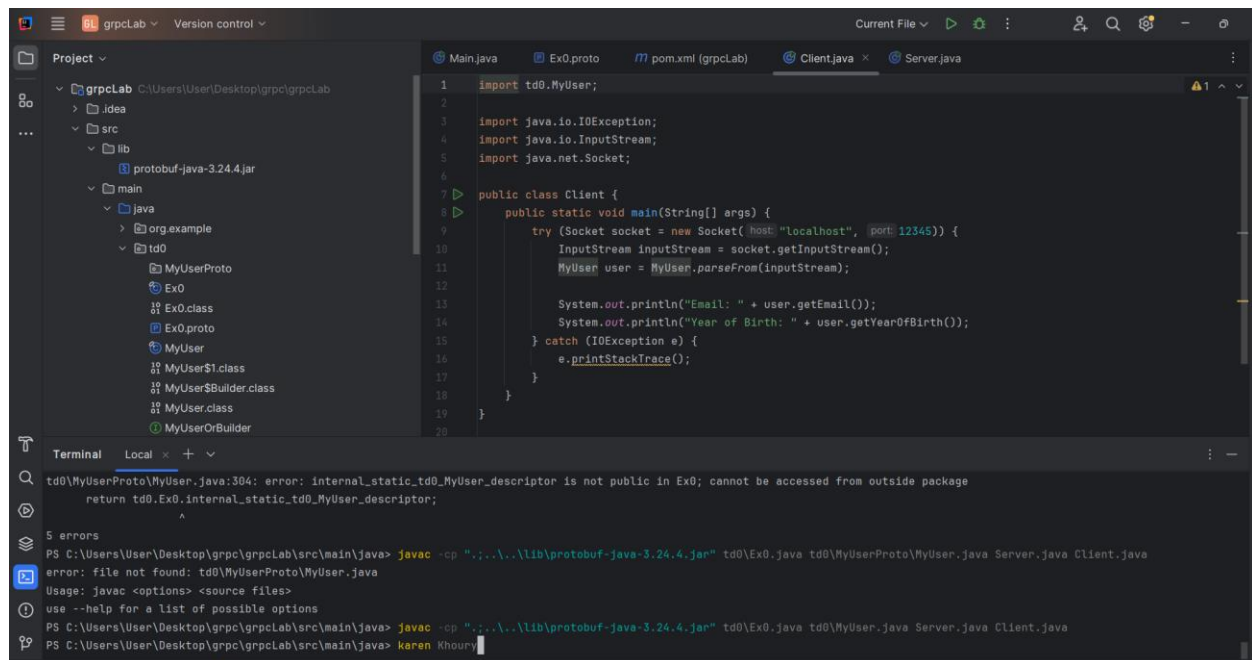


Editing pom.xml and reloading maven:

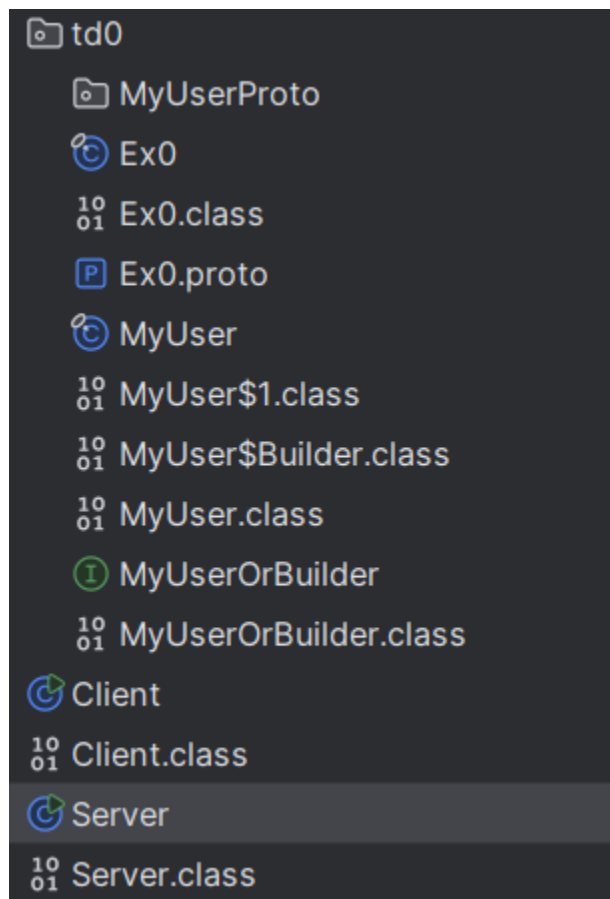


Run your server, then your client. Print-screen the output and explain:

```
javac -cp ".;..\lib\protobuf-java-3.24.4.jar" td0\Ex0.java td0\MyUser.java Server.java Client.java
```



output files:



Karen Khoury 220402

Terminal 1:

```
java -cp ".;..\lib\protobuf-java-3.24.4.jar" Server
```

```
PS C:\Users\User\Desktop\grpc\grpcLab\src\main\java> java -cp ".;..\lib\protobuf-java-3.24.4.jar" Server
Server is listening on port 12345
```

Terminal 2:

```
java -cp ".;..\lib\protobuf-java-3.24.4.jar" Client
```

```
Terminal  Local x Local (2) x + v
PS C:\Users\User\Desktop\grpc\grpcLab> cd C:\Users\User\Desktop\grpc\grpcLab\src\main\java
PS C:\Users\User\Desktop\grpc\grpcLab\src\main\java> java -cp ".;..\lib\protobuf-java-3.24.4.jar" Client
Email: john.doe@example.com
Year of Birth: 1980
PS C:\Users\User\Desktop\grpc\grpcLab\src\main\java> Karen khoury
```

Communication réussie entre 2 programmes Java utilisant Protobuf

Protobuf est un format binaire de sérialisation de données développé par Google, utilisé pour transmettre efficacement des objets structurés entre applications.

Ici, on a créé un objet MyUser, que le serveur a sérialisé avec Protobuf et envoyé via un socket, puis le client l'a reçu, désérialisé et affiché.

D'après ChatGPT :

Décryptage Protobuf :

◆ **0A = 00001010**

- **Tag = 1** (0001)
- **Wire type = 2** (010) → signifie une chaîne de caractères (length-delimited)
- Donc : **champ 1 = string (email)**

◆ **14 = 20 (longueur de la chaîne suivante)**

Ensuite, tu as :

```
r
CopyEdit
6a 6f 68 6e 2e 64 6f 65 40 65 78 61 6d 70 6c 65 2e 63 6f 6d
```

→ C'est "john.doe@example.com" en ASCII → ☒

◆ 10 = 00010000

- **Tag = 2** (0010)
- **Wire type = 0** (000) → type entier varint (typiquement `int32`)
- Donc : **champ 2 = int (année de naissance)**

◆ BC 0F

→ C'est un entier en varint → en base 10 ça donne :

```
bash
CopyEdit
BC = 0b10111100
0F = 0b00001111
```

→ Varint = 1980 ☒

Partie 2 :

```
protoc -I . --java_out . --grpc_out . --plugin=protoc-gen-grpc=protoc-gen-grpc-java-1.50.2-windows-x86_64.exe Lab1GRPC.proto
```

Explain each parameter of the command:

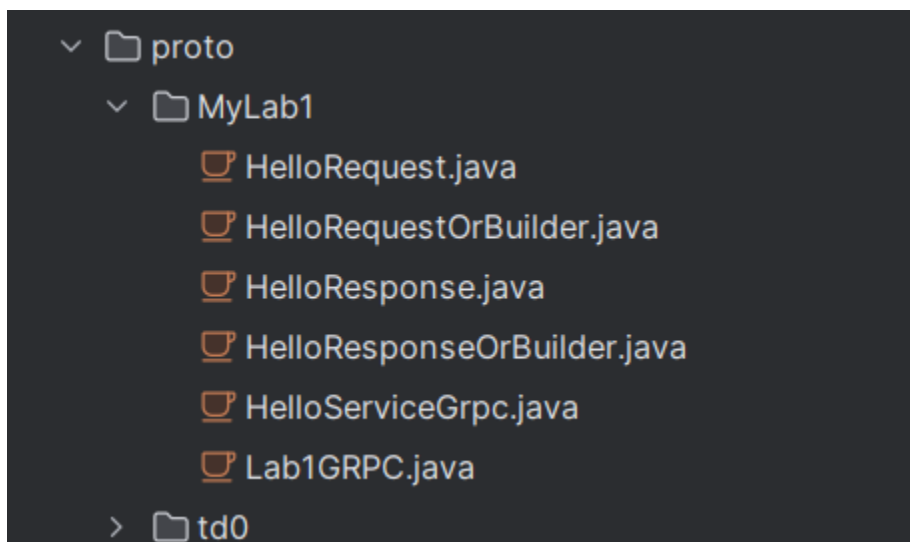
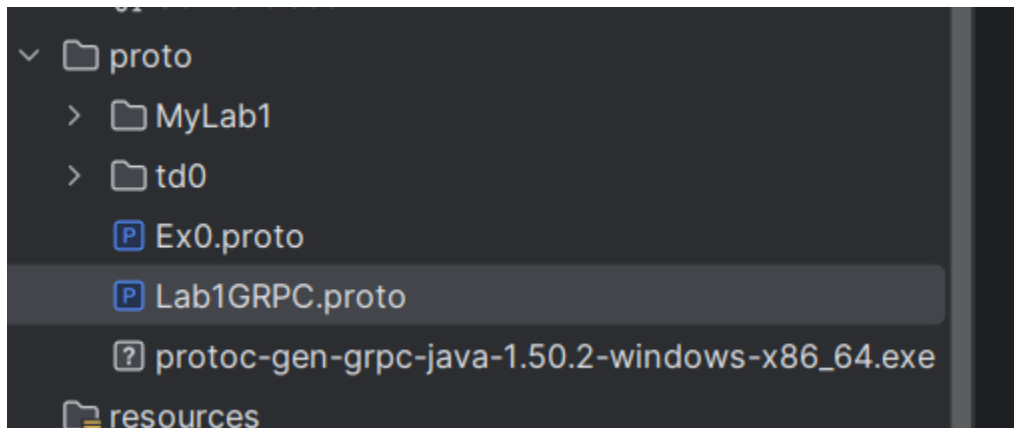
protoc: Compilateur protobuf

-I . : repertoire ou se trouve le fichier .proto (dossier courant)

--java_out . : dossier ou generer les classe java standard protobuf

--grpc_out : Dossier ou generer les stubs gRPC

--plugin : Lien vers le plugin grpc java (compi pour les stubs grpc)



La classe **HelloServiceImpl** implémente la méthode RPC sayHello, qui reçoit un message HelloRequest du client, extrait le message de salutation, puis construit une réponse HelloResponse personnalisée. Elle utilise un StreamObserver pour renvoyer la réponse au client et terminer proprement l'appel gRPC

La classe **HelloServer** démarre un serveur gRPC sur le port 50551, en y attachant le service HelloServiceImpl qui répond aux requêtes SayHello. Ce service reçoit un message contenant une salutation, construit une réponse avec "Hello, ..." et l'envoie au client

La classe **HelloClient** crée un canal de communication vers le serveur gRPC, envoie un message HelloRequest contenant une salutation, puis attend une réponse. Elle affiche ensuite la réponse HelloResponse envoyée par le serveur avec le message personnalisé.

>Server started on port 50551

>Réponse du serveur : Hello, Bonjour depuis le client 🙌 !